

name- AKSHAY CHAUHAN

CUID- CU24250262

SECTION- C

IMPORTING LIBRARIES

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully!")
```

Libraries imported successfully!

IMAGE READ AND DISPLAY

```
img = cv2.imread("image.jpg")

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

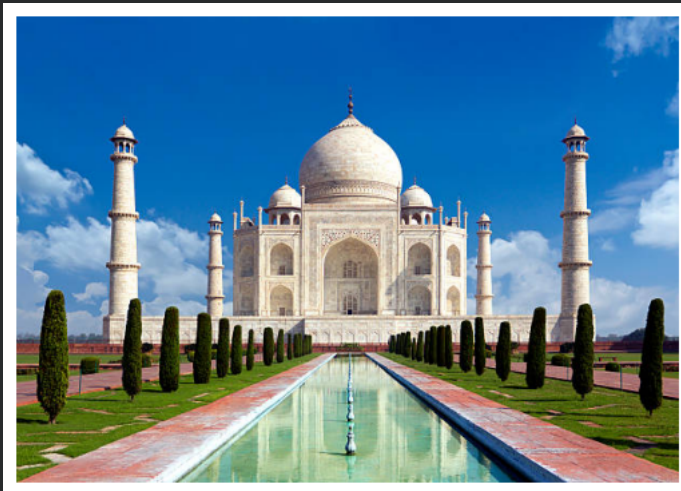


Image properties check

```
print("Image Shape:", img.shape)
print("Image Height:", img.shape[0])
print("Image Width:", img.shape[1])
print("Number of Channels:", img.shape[2])
print("Data Type:", img.dtype)
```

Image Shape: (431, 612, 3)
Image Height: 431
Image Width: 612
Number of Channels: 3
Data Type: uint8

**GRAYSCALE **

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap="gray")
plt.axis("off")
plt.show()
```



HSB COVERSION

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.axis("off")
plt.show()
```



LAB CONVERSION

```
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.axis("off")
plt.show()
```



RESIZING

```
resized = cv2.resize(img, (400, 300))

plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



ROTATION

```
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



HORIZONTAL FLIP

```
horizontal_flip = cv2.flip(img, 1)

plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



VERTICAL FLIP

```
vertical_flip = cv2.flip(img, 0)

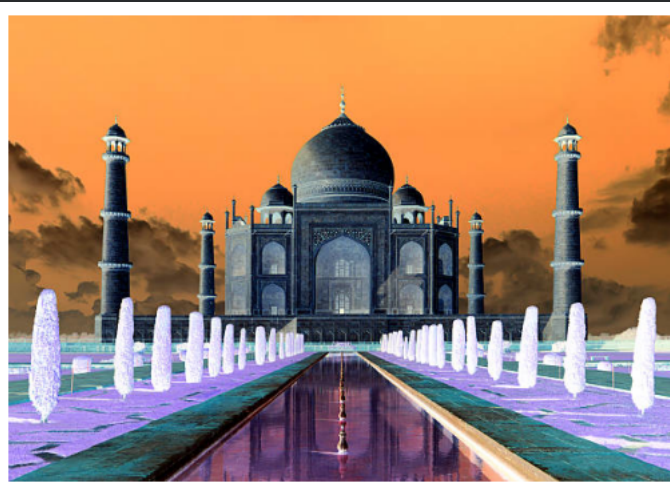
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



NEGATIVE IMAGE

```
negative = 255 - img

plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



ROI (Region of Interest)

```
roi = img[100:300, 100:300]

plt.imshow(cv2.cvtColor(ro, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

**IMAGE SAVE**

```
cv2.imwrite("output.jpg", img)
cv2.imwrite("output.png", img)

print("Images saved successfully!")
```

Images saved successfully!

Start coding or [generate](#) with AI.

QUESTIONS**Q1. What is a digital image? Differentiate between grayscale and color images.**

Answer:

A digital image is a two-dimensional array of pixels that represents a visual scene in a form that a computer can process. Each pixel contains numerical information about its brightness or color.

A grayscale image contains only brightness or intensity information. Each pixel represents a shade ranging from dark to bright, and it generally uses a single channel.

A color image contains color information and usually uses multiple channels to represent different colors, such as Red, Green, and Blue.

Therefore, grayscale images contain only intensity information, while color images contain both color and intensity information.

Q2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

Answer:

Different color spaces represent image colors in different ways.

RGB: Represents an image using Red, Green, and Blue channels. It is commonly used for displaying images. BGR: Uses Blue, Green, and Red channels. BGR is the default channel order used by OpenCV. HSV: Represents color using Hue, Saturation, and Value. It is useful when we need to identify or separate specific colors. LAB: Represents color using lightness and two color components. It is useful for analyzing color and brightness separately.

RGB and BGR mainly differ in the order of the color channels, while HSV and LAB represent color in different spaces

Q3. What is the purpose of converting an image to grayscale before further processing?

Answer:

An image is converted to grayscale to reduce the amount of information that needs to be processed. A grayscale image contains only intensity values instead of separate color channels.

This makes many image processing operations simpler and faster. It is especially useful when color information is not necessary and only brightness, edges, shapes, or textures are important.

Q4. Explain the concept of image complement (negative image) and mention its practical applications.

Answer:

An image complement, also called a negative image, is created by inverting the pixel intensity values of an image.

For an 8-bit image, the negative can be obtained using:

Negative Pixel = 255 – Original Pixel

As a result, dark regions become bright and bright regions become dark.

Applications:

Improving the visibility of certain image details. Analysing medical images. Highlighting features that may be difficult to see in the original image.

Q5. Differentiate between image resizing, cropping, and scaling.

Answer:

Resizing: Changes the dimensions of an image, such as changing it from 800×600 pixels to 400×300 pixels. Cropping: Removes unwanted parts of an image and keeps only the selected region. Scaling: Changes the size of an image by enlarging or reducing it while maintaining a proportional relationship between its dimensions.

In simple terms, resizing changes image dimensions, cropping selects a portion of the image, and scaling changes the size of the image

Q6. What is a Region of Interest (ROI), and why is it important in computer vision?

Answer:

A Region of Interest (ROI) is a selected portion of an image that contains the important information needed for further processing.

ROI is important because it allows a computer vision system to focus only on the relevant part of an image instead of processing the entire image.

This can:

Reduce unnecessary processing. Save computational resources. Make analysis more focused. Help in tasks such as object detection and image analysis.

The lab experiment specifically includes cropping an ROI and analysing its characteristics.

Q7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

Answer:

OpenCV is widely used for computer vision because it provides many ready-to-use functions for image processing and computer vision tasks.

It supports operations such as:

Reading and saving images. Color space conversion. Resizing and rotation. Image enhancement. Filtering and other computer vision operations.

It also works well with Python, NumPy, and other tools used for image processing. The lab specifically uses OpenCV along with NumPy and Matplotlib.

Q8. Explain how image resolution and pixel intensity influence image quality.

Answer:

Image resolution refers to the number of pixels used to represent an image. Higher resolution generally provides more detail because more pixels are available to represent the image.

Pixel intensity represents the brightness of a pixel. In a grayscale image, low intensity values represent darker pixels and high intensity values represent brighter pixels.

Therefore:

Higher resolution can provide more image detail. Proper pixel intensity distribution can improve brightness and contrast. Poor resolution or unsuitable intensity values can reduce image quality.

Q9. Mention five real-world applications where basic image preprocessing is an essential step.

Answer:

Five real-world applications are:

Medical Imaging – preprocessing helps improve the visibility of details in X-rays and other medical images. Autonomous Vehicles – images are preprocessed before detecting objects and road features. Surveillance Systems – preprocessing helps improve images before object or face detection. Robotics – robots preprocess camera images before analysing their surroundings. Industrial Automation – preprocessing is used before detecting defects or objects in manufactured products.

These types of applications are also highlighted in the lab sheet as areas where image processing forms an important foundation

Q10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Answer:

Image preprocessing improves the quality and consistency of input images before they are given to computer vision algorithms or deep learning models.

Techniques such as resizing, grayscale conversion, filtering, and other enhancement operations can:

Remove unwanted noise. Improve image quality. Highlight important features. Make images more consistent in size and format. Reduce unnecessary information.

As a result, feature extraction and deep learning models can focus more effectively on useful visual information. Preprocessing